

Técnicas de Reconocimiento de Patrones:

Uso de la arquitectura YOLOv5 para detección y clasificación de imágenes térmicas.

J. Bengoa B. Barański S. Izquierdo

17/02/2026

Índice

1 Introducción

2 Fundamentos de YOLO

- Descripción de la Neurona
- Topología de la Red
- Entrenamiento de la Red e Hiperparámetros

3 Metodología

4 Resultados

5 Conclusión

Índice

1 Introducción

2 Fundamentos de YOLO

- Descripción de la Neurona
- Topología de la Red
- Entrenamiento de la Red e Hiperparámetros

3 Metodología

4 Resultados

5 Conclusión

Introducción a la termografía (TIR)

Contexto:

- Visión artificial crítica en seguridad y conducción autónoma.
- Imágenes RGB fallan con mala iluminación (niebla, noche).
- **Solución:** Termografía infrarroja (TIR) → detecta radiación térmica.

Desafíos en TIR:

- Baja resolución (sensores caros), falta de color (monocanal).
- Texturas difusas y efecto halo.

Objetivo

Validar la arquitectura **YOLOv5** sobre el dataset Teledyne FLIR para detección en tiempo real.

Estado del Arte

- **Enfoque Clásico: HOG + SVM.**
 - Poca robustez ante variabilidad térmica.
- **Deep Learning (CNNs):**
 - ① **Dos etapas (R-CNN):**
 - Alta precisión.
 - Alta carga computacional.
 - Lento para tiempo real.
 - ② **Una etapa (YOLO):**
 - Predicción en una sola evaluación.
 - Equilibrio velocidad-precisión.



Figura: Imagen térmica de ejemplo.

Metodología YOLO

Divide la imagen en una rejilla $S \times S$. Es un problema de regresión para encontrar la posición del objeto y de clasificación del mismo.

Índice

1 Introducción

2 Fundamentos de YOLO

- Descripción de la Neurona
- Topología de la Red
- Entrenamiento de la Red e Hiperparámetros

3 Metodología

4 Resultados

5 Conclusión

La Neurona Convolutiva en YOLO

Unidad básica de procesamiento. Hiperparámetros clave: k (kernel), P (padding), S (stride).

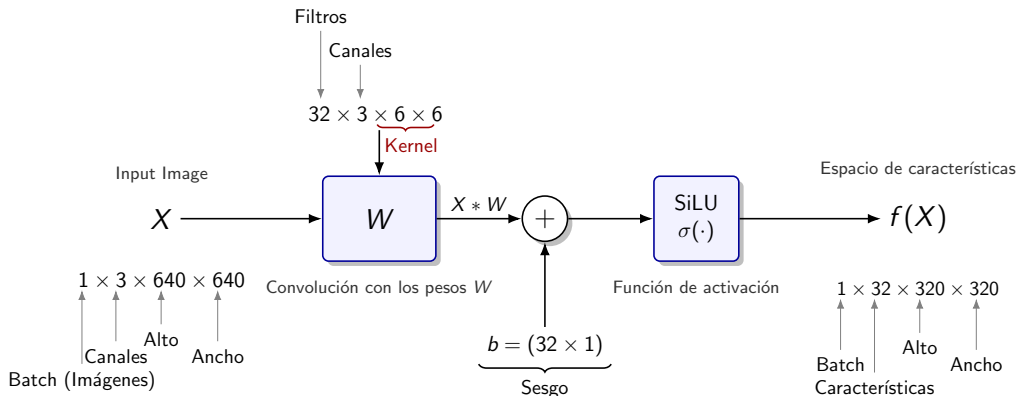


Figura: Estructura de la neurona a la entrada.

Ecuaciones del modelo: Convolución y Dimensiones

Operación de Convolución 2D:

- Implementada técnicamente como correlación cruzada (sin voltear el kernel):

$$(W * X)(c_{out}, h, w) = \sum_{l=0}^{C_{in}-1} \sum_{i=0}^{K_H-1} \sum_{j=0}^{K_W-1} W(c_{out}, l, i, j) \cdot X(l, h \cdot S + i \cdot d, w \cdot S + j \cdot d) \quad (1)$$

Dimensiones del mapa de características de salida:

- Relación generalizada considerando el factor de dilatación d :

$$Out = \left\lfloor \frac{In + 2P}{S} \right\rfloor + 1 \quad (2)$$

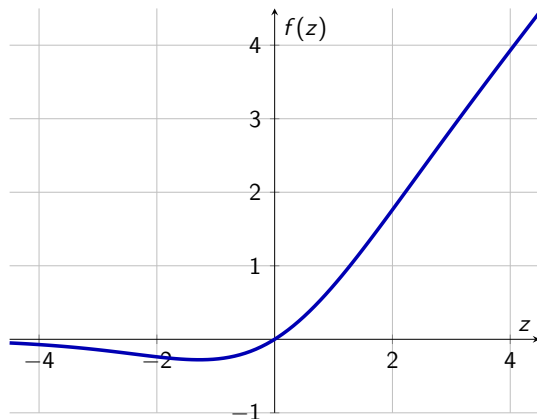
Función de Activación: SiLU

Sigmoid Linear Unit (SiLU):

$$f(z) = z \cdot \sigma(z) = \frac{z}{1 + e^{-z}} \quad (3)$$

- Permite flujo de gradientes en valores negativos pequeños.
- Suavidad superior a ReLU.
- Mejora la convergencia en redes profundas.

Gráfica de SiLU



Índice

1 Introducción

2 Fundamentos de YOLO

- Descripción de la Neurona
- Topología de la Red
- Entrenamiento de la Red e Hiperparámetros

3 Metodología

4 Resultados

5 Conclusión

Bloque ResNet (Residual Network)

- **Problema:** Gradientes explotan/desvanecen en redes muy profundas.
- **Solución:** Conexión identidad. $y = \mathcal{F}(x) + x$.

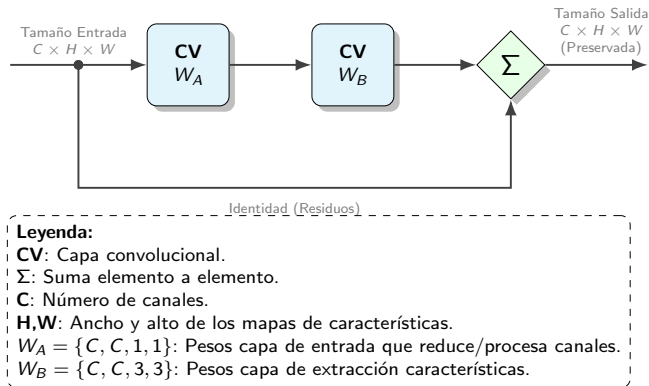


Figura: Arquitectura Bloque ResNet: Aprende $\mathcal{F}(x)$ (ruido/características) sobre la identidad.

Bloque CSP / C3 (Cross Stage Partial)

- **Problema:** Ineficiencia por duplicidad de información en el gradiente y cálculos redundantes en redes profundas.
- **Solución:** Bifurcar el flujo de entrada. Una rama extrae características complejas mientras la otra preserva el contexto global. $y = \mathcal{T}[\mathcal{F}(x) \parallel \mathcal{G}(x)]$

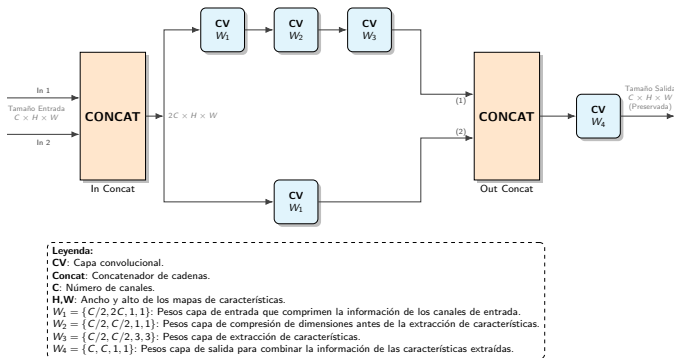


Figura: Bloque CSP: Mejora la variabilidad del aprendizaje reduciendo la redundancia.

Bloque C3-k (CSPResNe(X)t)

- **Problema:** Necesidad de aumentar la profundidad de la red para extraer características complejas sin sufrir degradación del gradiente.
- **Solución:** Sustituir convoluciones simples por una serie de k bloques **ResNet** en la rama profunda de la arquitectura CSP.

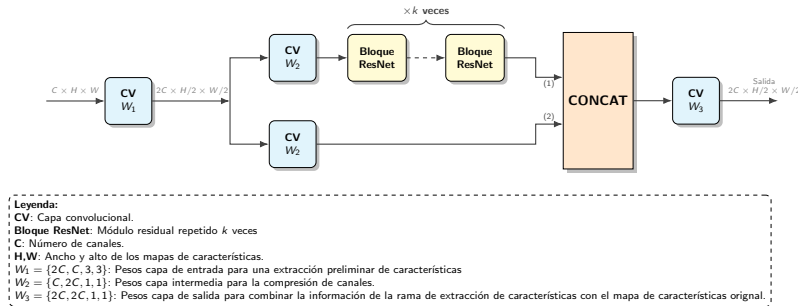


Figura: Bloque C3-k: Mejora del aprendizaje para no sufrir degradación del gradiente.

Bloque SPPF (Spatial Pyramid Pooling Fast)

- **Problema:** Capturar contexto multiescala sin el coste computacional de kernels grandes en paralelo.
- **Solución:** Estructura en **cascada** de Max Pooling (5×5). Tres capas en serie equivalen matemáticamente a un kernel grande.
- **Invarianza:** El Max Pooling extrae la característica dominante, otorgando robustez ante traslaciones y deformaciones del objeto.

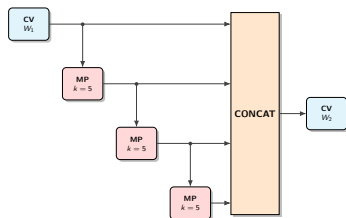


Figura: Arquitectura SPPF.

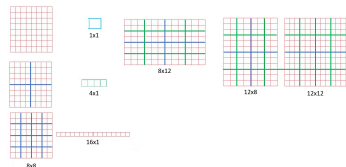
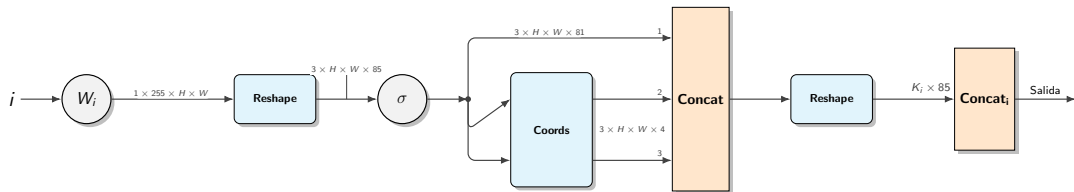


Figura: Ejemplo del comportamiento invariante.

Cabeza de Detección (Head)

- **Multiescala:** 3 ramas de detección ($i = \{1, 2, 3\}$) para diferentes tamaños de objeto.
- **Salidas por anclaje:**
 - 1 Coordenadas de la caja delimitadora (t_x, t_y, t_w, t_h).
 - 2 Probabilidad de objeto.
 - 3 Probabilidades de clasificación por categoría.
- **Arquitectura:** Red totalmente convolucional (FCN) sin capas densas.



Leyenda:

W_i : Convolución de tamaño $255 \times i \cdot 128 \times 1 \times 1$

σ : Función de activación Relu.

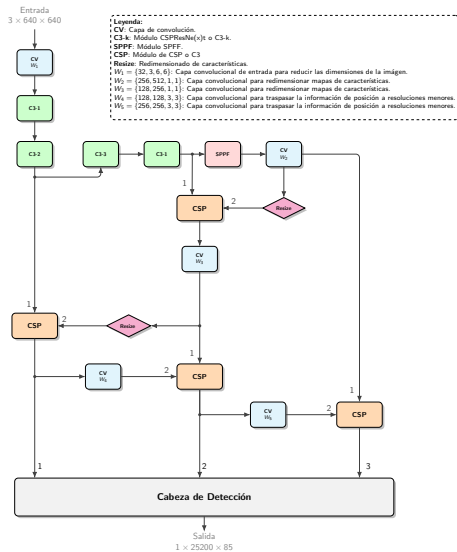
Reshape: Bloque que transforma las dimensiones de los tensores.

Coords: Bloque encargado de realizar las operaciones necesarias para transformar a coordenadas absolutas.

Concat: Concatenador de cadenas.

Figura: Esquema de la Cabeza de Detección: Procesamiento de tensores para predicción multiescala.

Arquitectura Global YOLO: Backbone + Neck + Head



Índice

- 1 Introducción
- 2 Fundamentos de YOLO
 - Descripción de la Neurona
 - Topología de la Red
 - Entrenamiento de la Red e Hiperparámetros
- 3 Metodología
- 4 Resultados
- 5 Conclusión

Función de Pérdida

Intuición (YOLOv1):

$$\begin{aligned}\mathcal{L} = & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{i,j}^{\text{obj}} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2] + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{i,j}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\ & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{i,j}^{\text{obj}} (C_i - \hat{C}_i)^2 + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{i,j}^{\text{noobj}} (C_i - \hat{C}_i)^2 + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2\end{aligned}$$

Refinamiento (YOLOv5):

$$\begin{aligned}\mathcal{L} &= \lambda_{\text{loc}} \mathcal{L}_{\text{loc}} + \lambda_{\text{obj}} \mathcal{L}_{\text{obj}} + \lambda_{\text{cls}} \mathcal{L}_{\text{cls}} \\ &= \lambda_{\text{loc}} (1 - \text{CIoU}) + \lambda_{\text{obj}} \text{BCE}_{\text{obj}} + \lambda_{\text{cls}} \text{BCE}_{\text{cls}}\end{aligned}$$

Detección mediante cajas delimitadoras

Detección (cajas delimitadoras):

$$b_x = [2\sigma(t_x) - 0,5] + c_x$$

$$b_y = [2\sigma(t_y) - 0,5] + c_y$$

$$b_w = p_w[2\sigma(t_w)]^2$$

$$b_h = p_h[2\sigma(t_h)]^2$$

Cajas predeterminadas (p_w, p_h)

Valores predefinidos de ancho y alto obtenidos mediante algoritmo **K-Means** sobre el set de entrenamiento.

- La red aprende factores de escala sobre estos moldes base en lugar de dimensiones absolutas desde cero.

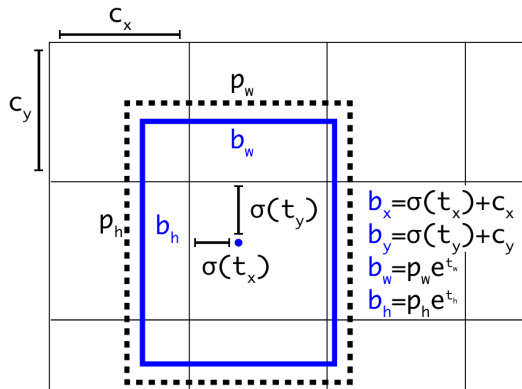


Figura: Esquema de predicción de coordenadas.

Actualización de los pesos

La actualización de pesos sigue la regla del **Descenso del Gradiente Estocástico**, usando **CLR** como *scheduler* para la tasa de aprendizaje ε :

$$\theta_{i+1} = \theta_i - \varepsilon \nabla_{\theta} \mathcal{L}(\theta_i)$$

1. Fase de Calentamiento (Warmup):

$$\varepsilon = \varepsilon_0 + \frac{n_i}{n_w}(\varepsilon_1 lr_0 - \varepsilon_0)$$

$$\varepsilon_1 = \left(1 - \frac{ep}{Ep}\right) (1 - lr_f) + lr_f$$

$$n_i = i + ep \cdot n_b$$

$$n_w = \max(\text{warmup_epochs} \cdot n_b, 100)$$

2. Fase de Entrenamiento:

$$\varepsilon = \varepsilon_0 \left[\left(1 - \frac{ep}{Ep}\right) (1 - lr_f) + lr_f \right]$$

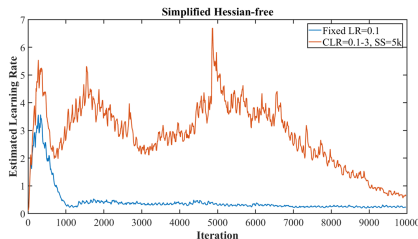


Figura: CLR (rojo) vs Fijo (azul).

Intuición

Usar CLR permite mantener tasas de aprendizaje altas. La convergencia es más rápida y evita mínimos locales.

Momento de Nesterov

lérmino de momento $\mu \cdot v_i$ para evitar mínimos locales y acelerar en zonas planas:

$$\begin{cases} \theta_{i+1} = \theta_i - \varepsilon v_{i+1} \\ v_{i+1} = \mu \cdot v_i + \nabla_{\theta} \mathcal{L}(\theta_i - \varepsilon \mu \cdot v_i) \end{cases}$$

Calendario de Actualización:

El momento crece linealmente durante el *warmup* hasta fijarse en momentum:

$$\mu = \frac{n_i}{n_w} (\text{mom} - \text{wup_mom}) + \text{wup_mom}$$

Regularización L2 (Weight Decay)

Penalización cuadrática de los pesos en la pérdida:

$$\mathcal{L}_{\text{reg}} = \mathcal{L}(\theta) + \frac{\text{wd}}{2} \sum_i \theta_i^2$$

Implementación Eficiente:

Se integra la penalización en la regla de actualización, evitando la suma de los cuadrados de los pesos (*forward*) y derivar el término del sumatorio (*backward*):

$$\theta_{i+1} = \theta_i(1 - \varepsilon \cdot \text{wd}) - \varepsilon \nabla_{\theta} \mathcal{L}(\theta_i)$$

Estrategias de Data Augmentation

degrees



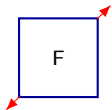
Gira la imagen aleatoriamente. Detecta objetos no verticales.

translate



Desplaza la imagen. Evita asociación por ubicaciones fijas.

scale



Zoom in/out. Reconocimiento a diferentes tamaños.

shear



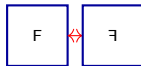
Distorsión oblicua. Simula estiramiento desde una esquina.

perspective



Transformación 3D. Simula cambios en el ángulo de la cámara.

flipud / fliplr



Efecto espejo vertical u horizontal.

mosaic



Varias imágenes en una. Contextos variados y escalas simultáneas.

mixup



Mezcla de dos imágenes con transparencia.

copy_paste



Pega objetos de una imagen en otra nueva.

Índice

- 1 Introducción
- 2 Fundamentos de YOLO
 - Descripción de la Neurona
 - Topología de la Red
 - Entrenamiento de la Red e Hiperparámetros
- 3 Metodología
- 4 Resultados
- 5 Conclusión

Preprocesamiento: De COCO a YOLO

- **Transformación de Coordenadas**

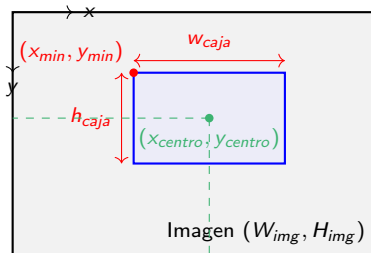
- **Entrada (COCO):** Esquina superior izq. (x_{min}, y_{min}) y dimensiones absolutas.
- **Salida (YOLO):** Centro (x_c, y_c) y dimensiones normalizadas $[0, 1]$.

$$x_c = x_{min} + w/2$$

$$y_c = y_{min} + h/2$$

Norm $\rightarrow$ dividir por W_{img}, H_{img}

- **Conversión de fichero JSON (COCO) a fichero txt (YOLO)**



Configuración del Entrenamiento y Hardware

Adaptación a Imágenes Térmicas

- Modificación de la capa de entrada: **1 canal** (Escala de grises).
- **Desactivación** de aumentos de color (HSV) para preservar la coherencia térmica.

Hiperparámetros (train.py)

- **Modelo Base:** YOLOv5s (Transfer Learning).
- **Épocas:** 500.
- **Batch size:** 64.
- **Resolución:** 640×640 .

Nodo HPC



NVIDIA A100

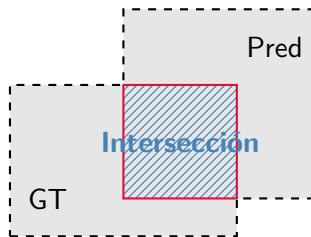
(Ampere)

192 GiB RAM

SSD M.2

Evaluación: Intersección sobre Unión (IoU)

La calidad de la localización se mide comparando el área de la predicción con la etiqueta real (Ground Truth).



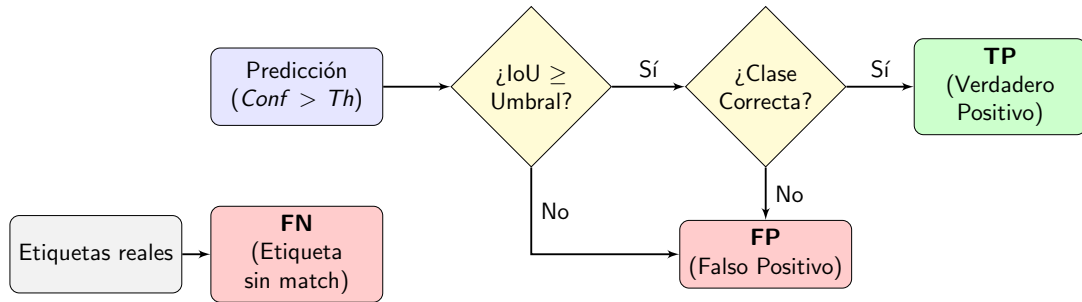
$$\text{IoU} = \frac{\text{Área Intersección}}{\text{Área Unión}}$$

$$\text{IoU} \in [0, 1]$$

$$\text{Unión} = \text{Área Total} - \text{Intersección}$$

Clasificación de Predicciones

Para construir la matriz de confusión, cada predicción se clasifica según su IoU y su clase.



Métricas de Desempeño (I): Evaluación Puntual

Precisión (P)

Fiabilidad de las detecciones positivas.

$$\text{Precisión} = \frac{TP}{TP + FP}$$

F1-Score

Media armónica entre Precisión y Recall (balance único).

$$F_1 = 2 \cdot \frac{\text{Precisión} \cdot \text{Recall}}{\text{Precisión} + \text{Recall}}$$

Recall (R)

Capacidad del modelo para encontrar todos los objetos reales.

$$\text{Recall} = \frac{TP}{TP + FN}$$

IoU Media

Calidad media de la localización geométrica de los aciertos.

$$\text{IoU media (TP)} = \frac{1}{TP} \sum \text{IoU}_{TP}$$

Métricas de Desempeño (II): AP y mAP

Average Precision (AP)

Área bajo la curva P-R (aproximación por sumatoria de rectángulos):

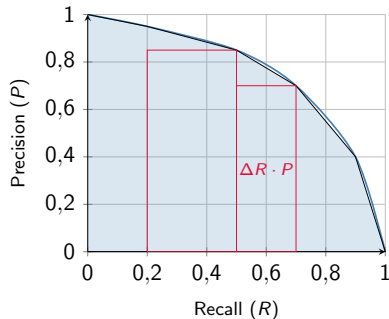
$$AP = \sum_k (R_{k+1} - R_k) \cdot P_{k+1}$$

mean Average Precision (mAP)

Promedio ponderado por la frecuencia de cada clase (n_c) para mitigar el desbalanceo:

$$mAP = \sum_{c=1}^N \underbrace{\frac{n_c}{\sum n_k}}_{w_c} \cdot AP_c$$

Cálculo del AP (Área bajo la curva)



Eficiencia y Visualización Cualitativa

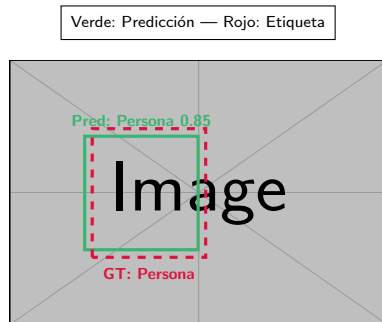
Eficiencia Computacional

$$FPS = \frac{\text{Total Imágenes}}{\text{Tiempo Procesamiento}}$$

Permite validar la viabilidad en tiempo real.

Visualización (video_process.py)

- Generación de vídeo a 25 FPS.
- Comparativa visual Frame a Frame.



Índice

- 1 Introducción
- 2 Fundamentos de YOLO
 - Descripción de la Neurona
 - Topología de la Red
 - Entrenamiento de la Red e Hiperparámetros
- 3 Metodología
- 4 Resultados
- 5 Conclusión

Análisis de Datos: Distribución de Clases

Limitación Crítica

Se observa un **severo desequilibrio** de clases en el conjunto de datos.

Impacto en el modelo:

- El aprendizaje quedará condicionado por las clases mayoritarias.
- Se favorece la predicción de *Coche* y *Persona*.
- Rendimiento deficiente esperado en clases con representación casi nula.

Clase	Train	Val	Test
Persona	50,478	4,470	12,323
Coche	73,623	7,133	30,517
Señal	20,770	2,472	5,660
Semáforo	16,198	2,005	6,758
Bicicleta	7,237	170	113
Moto	1,116	55	3,314
Camión	829	46	2,634
Autobús	2,245	179	0
Otro vehículo	1,373	63	696
Boca de incendios	1,095	94	277
Perro	4	0	25
Tren	5	0	0
Ciervo	8	0	0
Monopatín	29	3	0
Cochecito	15	6	0
Scooter	15	0	0
Total	175,040	16,696	62,317

Tabla: Distribución de muestras por conjunto.

Resultados de Inferencia: Predicciones vs. GT

Visualización de Resultados (YouTube)



▶ Haz clic para reproducir vídeo externo

- **Verde:** Detecciones del modelo.
- **Rojo:** Etiquetas reales.

Rendimiento en Test

4.3 FPS

Procesamiento por segundo

232.19

ms / imagen

Hardware Local

CPU: Intel Core i7-10510U

RAM: 8 GB DDR4

GPU: Intel UHD Graphics

Inferencia en CPU

Rendimiento Global del Modelo

- **Métricas globales:** Evolución de la precisión, recall, IOU y F1-score en función del umbral de confianza.
- **Bondad del clasificador:** Se obtiene un **MAP**= 0,475.

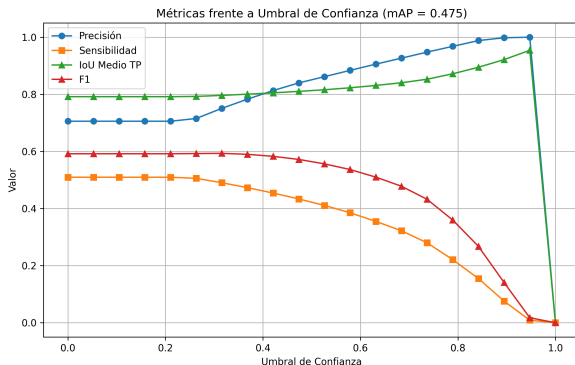


Figura: Curvas de métricas frente a la confianza para el conjunto de test.

Rendimiento Global del Modelo

- **Métricas globales:** Evolución de la precisión, recall, IOU y F1-score en función del umbral de confianza.
- **Bondad del clasificador:** Se obtiene un **MAP**= 0,475.

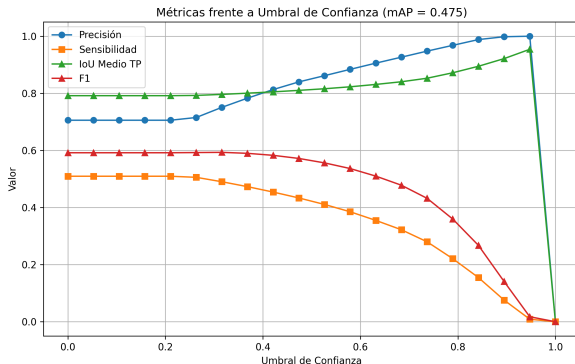


Figura: Curvas de métricas frente a la confianza para el conjunto de test.

Resultados Detallados por Categoría

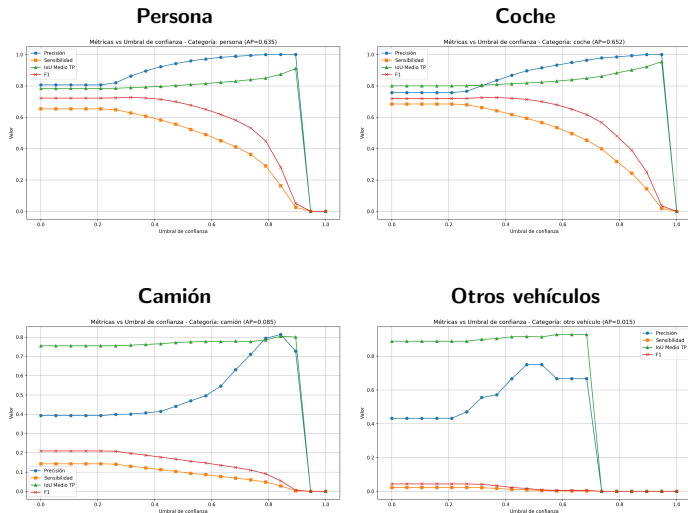


Figura: Métricas del clasificador algunas categorías del conjunto de datos.

Curva Precisión-Recall (PR)

- **Interpretación:** Evalúa el balance entre la exactitud de las detecciones y la capacidad del modelo para encontrar todos los objetos.

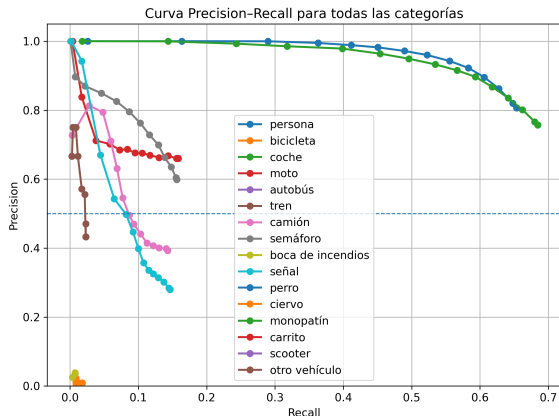


Figura: Curva PR global: El modelo ideal tendería hacia la esquina superior derecha.

Índice

- 1 Introducción
- 2 Fundamentos de YOLO
 - Descripción de la Neurona
 - Topología de la Red
 - Entrenamiento de la Red e Hiperparámetros
- 3 Metodología
- 4 Resultados
- 5 Conclusión

Validación de Arquitectura

Se demuestra la viabilidad de adaptar YOLOv5 (RGB) al espectro **infrarrojo**, centrando el aprendizaje en la morfología y huella térmica.

- **Rendimiento Cuantitativo:**

- mAP global de **0.475** (comparable al estado del arte YOLOv4).
- Alta precisión ($AP > 0,6$) en clases críticas: **Coche** y **Persona**.

- **Eficiencia Computacional:**

- Arquitectura de una sola etapa validada.
- 4.3 FPS en CPU → Escalabilidad garantizada para tiempo real en hardware dedicado (GPU/FPGA).

Limitaciones y Líneas Futuras

Limitaciones Críticas

- **Desbalanceo de Datos:** La detección es casi nula en clases minoritarias (bicicletas, perros) debido a la hegemonía de coches y personas.
- **Ambigüedad Térmica:** Falsos positivos en *Señales* por falta de textura/color.
- **Seguridad:** Se detectaron fallos en peatones vulnerables (niños). No apto aún para autonomía completa sin fusión de sensores.

Propuestas de Trabajo Futuro:

- ① **Remuestreo:** Técnicas para equilibrar la distribución de clases.
- ② **Agrupación Semántica:** Fusión de etiquetas de vehículos grandes.
- ③ **Información Temporal:** Arquitecturas híbridas para evitar pérdidas de detección en fotogramas consecutivos.

¿Preguntas?

Gracias por su atención